



Portail Étudiant

Plateforme de cours en ligne

M2 Lead Développeur Full Stack · 2025-2026

Accueil / **Projet Intégrateur — EventFlow**

EventFlow — Plateforme de Gestion d'Événements

Projet intégrateur · Symfony · Vue.js · Node.js · RGPD

Symfony 6+

Vue.js 3

Node.js / Vite

RGPD

API REST + JWT

EventFlow est une application web de gestion d'événements professionnels (conférences, meetups, formations). Les organisateurs créent et publient des événements ; les participants s'inscrivent. Le tout repose sur une architecture moderne — API Symfony + SPA Vue.js — avec une **conformité RGPD native et complète**.

 Binômes (2 étudiants)

 5 jours

 100 points + bonus

 Soutenance 25 min

 5 livrables

 Git obligatoire

Stack Technique & Couverture par Matière



Symfony 6+

- API REST (controllers, routes)
- Doctrine ORM + migrations
- JWT (LexikJWTBundle)
- Validator + Serializer
- Services & DI Container
- Events & Listeners
- Commandes console



Vue.js 3 + Node.js

- Composition API
- Vue Router (SPA)
- Pinia (store global)
- Axios (appels API)
- Vite (bundler)
- Guards de navigation
- Composants réutilisables



RGPD

- Consentement explicite
- Bandeau cookies granulaire
- Droit de rectification
- Anonymisation automatique
- Logs d'accès aux données
- Registre des traitements
- Politique de confidentialité

💡 **Architecture imposée** : dossier `/backend` (Symfony — API REST JSON) + dossier `/frontend` (Vue.js — SPA). Communication exclusivement via l'API. Un seul repo Git avec les deux projets.

Modèle de Données

Les 4 entités Doctrine à implémenter. Les champs marqués **RGPD** sont soumis à des règles de traitement spécifiques.

User

int	id
RGPD	email
string	password (hashé)
RGPD	firstName
RGPD	lastName
RGPD	phone (nullable)
enum	role
RGPD	consentDate
string	consentVersion
bool	isAnonymized
datetime	createdAt

Event

int	id
string	title
text	description
datetime	eventDate
string	location
int	maxParticipants
ManyToOne	organizer → User
bool	isPublished
datetime	createdAt

Registration

int id

ManyToOne user → User

ManyToOne event → Event

datetime registeredAt

enum status

Status : pending / confirmed / cancelled

ConsentLog

int id

ManyToOne user → User

enum action

datetime timestamp

RGPD ipAddress (hashée)

string details

Actions : consent_given · consent_withdrawn ·
data_accessed · data_deleted

Fonctionnalités à Développer

 **Fonctionnalités Symfony — Backend API**



 **Fonctionnalités Vue.js — Frontend SPA**



 **Fonctionnalités RGPD — Conformité native**



Planning des 5 Jours

Chaque jour est organisé en demi-journées. La répartition des tâches entre les deux membres du binôme est libre — mais le Git doit montrer des contributions équilibrées.

Jour 1 — Setup & Architecture

- **Sf** Init projet
Symfony, config BDD
- **Sf** Entités +
migrations Doctrine
- **Sf** JWT : register +
login endpoints

Jour 2 — Core Features

- **Sf** CRUD
Événements (API)
- **Sf** Endpoint
inscription à un
événement
- **Sf** Gestion des rôles
(Voter)

Jour 3 — RGPD

- **RGPD** Bandeau
cookies (3 catégories)
- **RGPD** ConsentLog
+ Listener Symfony
- **RGPD** Page "Mes
données" (profil)

- **Vue** Init Vue 3 + Vite + Pinia
- **Vue** Vue Router + routes de base
- **Vue** Pages Login / Register
- **RGPD** Champ consentDate + version

- **Vue** Liste + détail événements
- **Vue** Dashboard organisateur
- **Vue** Guards de navigation
- **Vue** Formulaire création événement

- **RGPD** Rectification + anonymisation
- **Sf** Commande anonymize-old-users
- **Vue** Page politique confidentialité
- **RGPD** Hash IP dans ConsentLog

Jour 4 — Qualité

- **Sf** Validation Symfony (Assert)
- **Sf** Codes HTTP + gestion erreurs
- **Vue** Validation formulaires front
- **Vue** Responsive + UX (feedback)
- **Sf** Tests PHPUnit (bonus)
- **Vue** Collection Postman complète
- **Sf** Documentation API (README)

Jour 5 — Finalisation

- Correction des derniers bugs
- README final (install + lancement)
- **RGPD** Registre des traitements
- Vérification Git propre
- Préparation démo live
- Répétition soutenance
- 🗣 Soutenances

💡 **Organisation recommandée** : Jours 1 et 2 en parallèle (un membre sur le backend Symfony, l'autre sur le frontend Vue.js). Jour 3 ensemble sur le RGPD — c'est le cœur du projet et ça nécessite de coordonner les deux couches. Jours 4 et 5 en intégration commune.



Les 5 Livrables

1

Code source complet — repo Git **Obligatoire**

Un seul dépôt Git avec deux dossiers : **/backend** (Symfony) et **/frontend** (Vue.js). Historique de commits propre, branches feature si possible. Les deux membres doivent apparaître dans les commits.

2

README technique complet

Prérequis (PHP, Node, Composer, npm), variables d'environnement nécessaires (`.env.example`), commandes d'installation et de lancement pas à pas, architecture du projet expliquée en quelques lignes.

3

Registre des traitements RGPD

Document (PDF ou Markdown) listant les

8 traitements identifiés

avec pour chacun : finalité, base légale, données concernées, destinataires, durée de conservation, mesures de sécurité. Le DPO fictif doit être désigné.

4

Collection Postman (ou fichier .http)

Tous les endpoints testés avec les requêtes et réponses exemples. Inclure les cas d'erreur (401, 403, 404, 422). Doit pouvoir être importée et exécutée directement.

5

Soutenance orale — 15 min démo + 10 min Q&A

Démonstration live de l'application (pas de slides). Parcours complet : inscription → consentement → création d'un événement → inscription → page "mes données" → anonymisation. Questions techniques individuelles.



Grille de Notation



Symfony / Backend 35 pts

API REST fonctionnelle et complète (CRUD + auth JWT)	/ 15 pts
Architecture propre (Services, Repository, DTOs)	/ 10 pts
Validation, gestion des erreurs, codes HTTP corrects	/ 5 pts
Commande d'anonymisation fonctionnelle	/ 5 pts



Vue.js / Frontend 30 pts

SPA fonctionnelle (Vue Router + Pinia + Axios)	/ 12 pts
Composants réutilisables et code organisé	/ 8 pts
UX soignée (formulaires, feedback, responsive)	/ 6 pts
Guards de navigation (routes protégées par rôle)	/ 4 pts



RGPD 25 pts

Consentement implémenté (bandeau + BDD + granularité)	/ 8 pts
Droits utilisateurs (rectification + anonymisation)	/ 7 pts

Registre des traitements complet et cohérent

/ 6 pts

Logs d'accès + politique de confidentialité

/ 4 pts

Documentation & Soutenance 10 pts

README clair + installation qui fonctionne

/ 4 pts

Qualité et fluidité de la démonstration

/ 3 pts

Pertinence des réponses aux questions techniques

/ 3 pts

Bonus

+3 pts Tests PHPUnit (au minimum 5 tests : endpoints auth + CRUD + anonymisation)

+2 pts Docker Compose fonctionnel (PHP + Node + MySQL + Nginx) avec `docker-compose up` qui lance tout

+2 pts Export des données personnelles — droit de portabilité (GET /api/me/export → JSON ou CSV)

+2 pts CI/CD basique : GitHub Actions qui lance les tests PHPUnit à chaque push

Barème de conversion

17–20

85–100 pts

14–16

70–84 pts

11–13

55–69 pts

8–10

40–54 pts

0–7

< 40 pts

Questions Potentielles en Soutenance

Les questions sont posées individuellement à chaque membre du binôme. Chacun doit être capable de défendre l'ensemble du projet.

Questions Symfony

- Décrivez le cycle de vie d'une requête HTTP dans Symfony.
- Qu'est-ce que le Dependency Injection Container ? Donnez un exemple concret dans votre code.
- Comment avez-vous sécurisé vos endpoints ? Qu'est-ce qu'un Voter Symfony ?
- Quelle est la différence entre une Entity et un DTO ? Pourquoi utiliser

Questions Vue.js / Node.js

- Quelle est la différence entre `ref()` et `reactive()` en Vue 3 ?
- Comment fonctionne Pinia ? Pourquoi l'avoir choisi plutôt que Vuex ?
- Expliquez le principe d'un guard de navigation. Comment avez-vous protégé vos routes ?
- Qu'est-ce que Vite apporte par rapport à Webpack ?

des groupes de sérialisation ?

- Comment fonctionne votre commande d'anonymisation ? Comment la planifieriez-vous en production ?
- Qu'est-ce qu'un Event Subscriber Symfony ? Où l'avez-vous utilisé ?

- Comment gérez-vous le token JWT côté frontend (stockage, rafraîchissement, expiration) ?
- Qu'est-ce qu'un composable Vue 3 ? En avez-vous créé ?



Questions RGPD

- Sur quelle base légale repose le traitement principal de votre application ?
- Quelle est la différence entre anonymisation et pseudonymisation ? Qu'avez-vous implémenté ?
- Pourquoi conservez-vous le ConsentLog après l'anonymisation d'un compte ?
- Quels sont les délais de conservation que vous avez retenus et pourquoi ?
- Pourquoi hacher l'adresse IP plutôt que de la stocker en clair ?
- Qu'est-ce que le Privacy by Design ? Comment s'applique-t-il dans votre projet ?



Contraintes & Exigences Techniques

Points éliminatoires :

- Application qui ne se lance pas (README non fonctionnel) → note plafonnée à 8/20
- Aucune fonctionnalité RGPD implémentée → note plafonnée à 12/20
- Un seul membre du binôme visible dans les commits Git → -5 pts
- Mots de passe en clair dans la base de données → -10 pts

Contrainte

Exigence

Versionnage

Git obligatoire, repo partagé, commits réguliers avec messages descriptifs

Contrainte	Exigence
Sécurité	Mots de passe hashés (bcrypt), JWT signé, pas de données sensibles en clair dans le code
Variables d'env.	Fichier <code>.env.example</code> fourni, jamais de <code>.env</code> commité
Base de données	MySQL 8+ ou PostgreSQL 14+ avec migrations Doctrine versionnées
Versions	PHP 8.2+, Symfony 6.4+, Vue.js 3.x, Node.js 20 LTS
Communication	Le frontend ne doit jamais accéder directement à la BDD — tout passe par l'API

Ce projet évalue en une seule réalisation

Vue.js / Node	SPA, Composition API, Pinia, Vue Router, guards, Vite, composants réutilisables
Symfony	API REST, Doctrine ORM, JWT, Validator, Services, Events, commandes console
RGPD	Consentement, droits utilisateurs, anonymisation, registre des traitements, Privacy by Design

Bon projet à tous les binômes ! La qualité du code prime sur la quantité des fonctionnalités.